

Human Development Index (HDI) Predictor

Project Description:

The Human Development Index (HDI) Predictor is a Machine Learning based web application developed using Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn. The system predicts the Human Development Index (HDI) of a country using four major development indicators: Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) Per Capita. The application provides a simple and user-friendly interface where users can enter these values and instantly receive the predicted HDI score along with its corresponding development category.

The project uses a Linear Regression model trained on the Human Development Index dataset. The trained model is serialized using Pickle (.pkl) and integrated into a Flask web application for real-time predictions. The project also includes data preprocessing, exploratory data analysis (EDA), and visualization to understand relationships among development indicators and improve model performance.

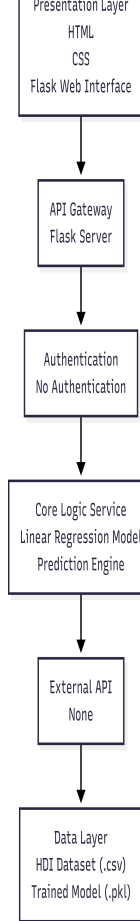
Scenarios

Scenario 1: A developed country with high life expectancy, high educational attainment, and high GNI per capita is entered into the system. The model predicts a Very High Human Development score.

Scenario 2: A developing country with moderate education levels, average life expectancy, and moderate income is evaluated. The application predicts a Medium Human Development score, helping identify areas for improvement.

Scenario 3: A country with low life expectancy, fewer years of schooling, and low GNI per capita is analyzed. The model predicts a Low Human Development score, supporting policy analysis and development planning.

Technical Architecture:



Description: The Human Development Index (HDI) Predictor follows a simple three-layer architecture consisting of the Presentation Layer, Application Layer, and Data Layer. Users provide input through the Flask web interface by entering Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) Per Capita. The Flask application processes the request using a trained Linear Regression model and predicts the Human Development Index (HDI). The predicted score is then classified into Low, Medium, High, or Very High Human Development and displayed on the web page.

Note: This project does not use a relational database. The application reads data from the Human Development Index (HDI) dataset in CSV format and uses a trained Linear Regression model (hdi_model.pkl) for prediction. Therefore, an ER Diagram is not required.

Pre-requisites:

- **Python Programming Knowledge:** <https://docs.python.org/3/>
- **Flask Framework:** <https://flask.palletsprojects.com/>
- **Machine Learning (Scikit-learn):** <https://scikit-learn.org/>
- **Pandas Library:** <https://pandas.pydata.org/>
- **NumPy Library:** <https://numpy.org/>
- **Matplotlib & Seaborn:** <https://matplotlib.org/> | <https://seaborn.pydata.org/>
- **Visual Studio Code:** <https://code.visualstudio.com/>
- **Git & GitHub:** <https://git-scm.com/> | <https://github.com/>

Project Workflow: The Human Development Index (HDI) Predictor project is developed through a structured workflow consisting of multiple milestones. Each milestone focuses on a specific phase of the Machine Learning application, from dataset preparation to deployment. This approach ensures systematic development, testing, and integration of all project components.

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Collect the Human Development Index (HDI) dataset from a reliable source.
- **Activity 1.2:** Analyze the dataset and select the Linear Regression algorithm for HDI prediction..
- **Activity 1.3:** Design the system architecture, including the Flask web interface, prediction model, and dataset integration.
- **Activity 1.4:** Configure the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, and other required libraries.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Perform data preprocessing, handle missing values, and prepare the dataset for training..
- **Activity 2.2:** Train the Linear Regression model, evaluate its performance, and save the trained model using Joblib.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the Flask backend by implementing application routes, loading the trained model, accepting user input, and generating HDI predictions.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the user interface using HTML and CSS.
- **Activity 4.2:** Integrate the frontend with the Flask backend and display prediction results dynamically.

Milestone 5: Deployment

- **Activity 5.1:** Configure the project for local execution and install all required dependencies.
- **Activity 5.2:** Run and test the Flask application locally to verify prediction accuracy and application functionality.

Milestone 6: Conclusion

Summarize the project outcomes, evaluate the prediction model, and identify future improvements such as enhanced algorithms, larger datasets, and cloud deployment.

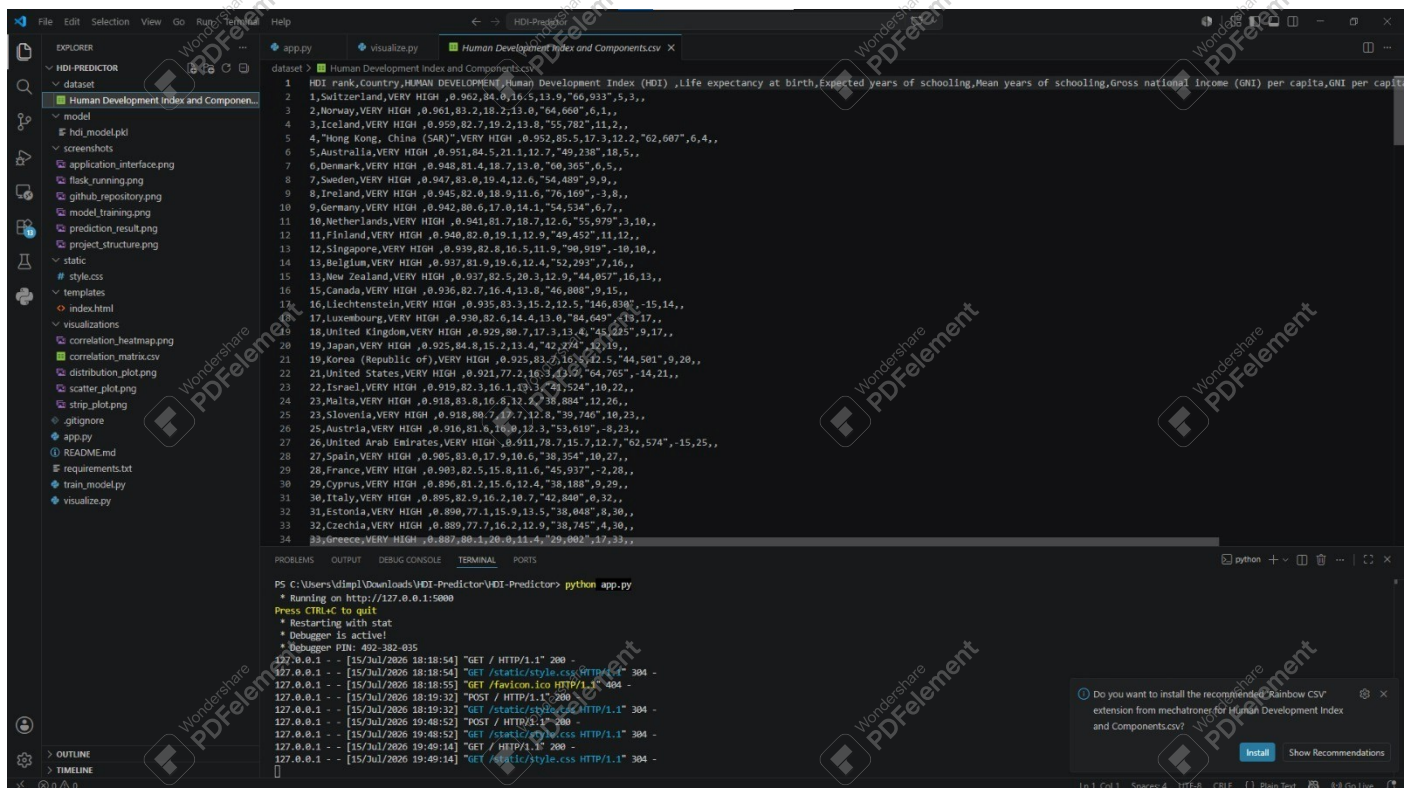
Milestone 1: Model Selection and Architecture

Before building the prediction model, a suitable Human Development Index dataset was collected from an open-source repository. The dataset contains important development indicators that influence the Human Development Index of different countries.

Activity 1.1: Dataset Collection and Understanding

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Download the Dataset:** Download the Dataset: Download the Human Development Index dataset in CSV format from a trusted source.
- **Step 2 — Explore the Dataset:** Open the dataset using Microsoft Excel or Pandas to examine the available columns and records.
- **Step 3 — Understand the Features:** Identify the important attributes used for prediction, including: Human Development Index (HDI), Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, Gross National Income (GNI) Per Capita
- **Step 4 — Identify the Target Variable:** Select HDI as the target variable and use the remaining indicators as input features for model training.
- **Step 5 — Save the Dataset:** Store the dataset inside the dataset folder of the project for preprocessing and model development.



```

dataset > Human Development Index and Components.csv
1 HDI, Country, HUMAN DEVELOPMENT INDEX (HDI), Life expectancy at birth, Expected years of schooling, Mean years of schooling, Gross national income (GNI) per capita, GNI per capita
2 1, Switzerland, VERY HIGH, 0.962, 84.0, 16.5, 13.9, "66,937", 5,3,,
3 2, Norway, VERY HIGH, 0.961, 83.2, 18.2, 13.8, "64,660", 6,1,,
4 3, Iceland, VERY HIGH, 0.959, 82.7, 19.2, 13.8, "55,782", 11,2,,
5 4, "Hong Kong, China (SAR)", VERY HIGH, 0.952, 85.5, 17.3, 12.2, "62,607", 6,4,,
6 5, Australia, VERY HIGH, 0.951, 84.5, 21.1, 12.7, "49,238", 18,5,,
7 6, Denmark, VERY HIGH, 0.948, 81.4, 18.7, 13.0, "60,365", 6,5,,
8 7, Sweden, VERY HIGH, 0.947, 83.0, 19.4, 12.6, "54,489", 9,9,,
9 8, Ireland, VERY HIGH, 0.945, 82.0, 18.9, 11.6, "70,189", 3,8,,
10 9, Germany, VERY HIGH, 0.942, 80.6, 17.0, 14.1, "54,534", 6,7,,
11 10, Netherlands, VERY HIGH, 0.941, 81.7, 18.7, 12.6, "55,979", 3,10,,
12 11, Finland, VERY HIGH, 0.940, 82.0, 19.1, 12.9, "49,452", 11,12,,
13 12, Singapore, VERY HIGH, 0.939, 82.8, 16.5, 11.9, "90,919", 10,10,,
14 13, Belgium, VERY HIGH, 0.937, 81.9, 19.6, 12.4, "52,293", 7,16,,
15 14, New Zealand, VERY HIGH, 0.937, 82.5, 20.3, 12.9, "44,057", 16,13,,
16 15, Canada, VERY HIGH, 0.936, 82.2, 16.4, 13.0, "46,080", 9,15,,
17 16, Liechtenstein, VERY HIGH, 0.935, 83.3, 15.2, 12.5, "146,838", 15,14,,
18 17, Luxembourg, VERY HIGH, 0.930, 82.6, 14.4, 13.0, "84,649", 19,17,,
19 18, United Kingdom, VERY HIGH, 0.929, 80.7, 17.3, 13.4, "45,225", 9,17,,
20 19, Japan, VERY HIGH, 0.925, 84.8, 15.2, 13.4, "42,274", 12,19,,
21 20, Korea (Republic of), VERY HIGH, 0.925, 83.7, 16.5, 12.5, "44,501", 9,20,,
22 21, United States, VERY HIGH, 0.921, 77.2, 16.3, 13.9, "64,765", 14,21,,
23 22, Israel, VERY HIGH, 0.919, 82.3, 16.1, 13.1, "49,524", 10,22,,
24 23, Malta, VERY HIGH, 0.918, 83.8, 16.8, 12.2, "38,884", 12,26,,
25 24, Slovenia, VERY HIGH, 0.918, 80.7, 17.7, 12.8, "39,740", 10,23,,
26 25, Austria, VERY HIGH, 0.916, 81.6, 16.0, 12.3, "53,619", 8,23,,
27 26, United Arab Emirates, VERY HIGH, 0.911, 78.7, 15.7, 12.7, "62,574", 15,25,,
28 27, Spain, VERY HIGH, 0.905, 83.0, 17.9, 10.6, "38,354", 10,27,,
29 28, France, VERY HIGH, 0.903, 82.5, 15.8, 11.6, "45,937", 2,28,,
30 29, Cyprus, VERY HIGH, 0.896, 81.2, 15.6, 12.4, "38,188", 9,29,,
31 30, Italy, VERY HIGH, 0.895, 82.9, 16.2, 10.7, "42,840", 9,32,,
32 31, Estonia, VERY HIGH, 0.890, 77.1, 15.0, 13.5, "38,048", 8,30,,
33 32, Czechia, VERY HIGH, 0.889, 77.7, 16.2, 12.9, "38,745", 4,30,,
34 33, Greece, VERY HIGH, 0.887, 80.1, 20.0, 11.4, "29,002", 17,33,,
  
```

```

PS C:\Users\vipal\Downloads\VDI-Predictor> python app.py
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 492-382-035
127.0.0.1 - - [15/Jul/2026 18:18:54] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 18:18:54] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [15/Jul/2026 18:18:55] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [15/Jul/2026 18:19:32] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 18:19:32] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [15/Jul/2026 19:40:52] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 19:40:52] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [15/Jul/2026 19:40:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 19:49:14] "GET /static/style.css HTTP/1.1" 304 -
  
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Different regression algorithms were studied to determine the most suitable model for HDI prediction. Based on simplicity, performance, and prediction accuracy, Linear Regression was selected.

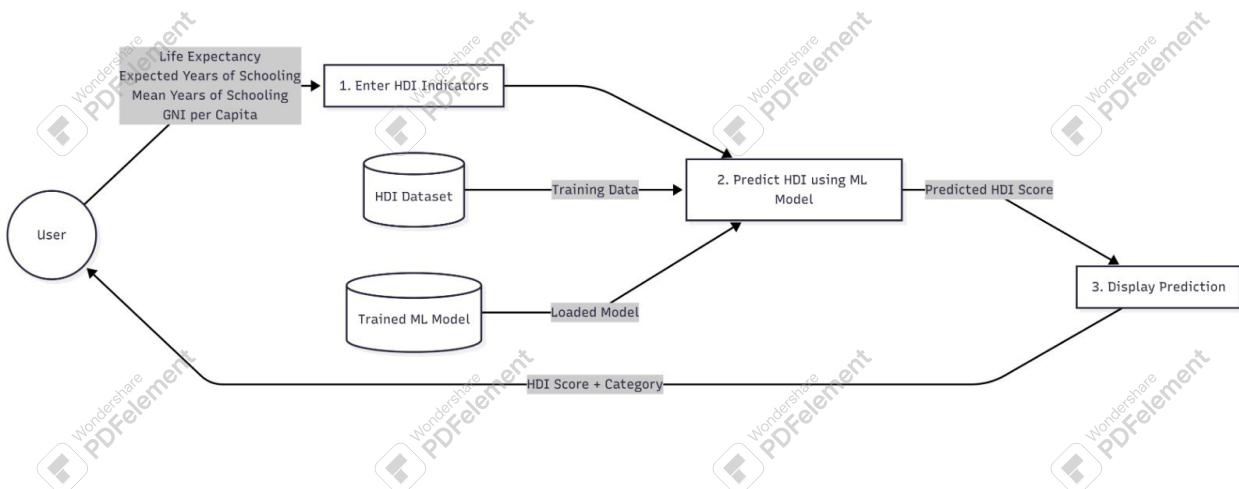
```
app.py visualize.py train_model.py X
train_model.py > ...
42 # Features
43 X = df[["Life", "Expected", "Mean", "GNI"]]
44
45 # Target
46 y = df["HDI"]
47
48 # Split
49 X_train, X_test, y_train, y_test = train_test_split(
50     X,
51     y,
52     test_size=0.2,
53     random_state=42
54 )
55
56 # Train
57 model = LinearRegression()
58 model.fit(X_train, y_train)
59
60 # Accuracy
61 pred = model.predict(X_test)
62
63 print("\nR2 Score:", round(r2_score(y_test, pred), 4))
64
65 # Save model
66 joblib.dump(model, "model/hdi_model.pkl")
67
68 print("\nModel Saved Successfully!")
69
```

Activity 1.3: Define the Architecture of the Application

The architecture was designed to illustrate how users interact with the Flask application, how the Machine Learning model processes input data, and how prediction results are displayed

Steps

- Design the system architecture.
- Define frontend and backend communication.
- Integrate the trained model.
- Display prediction results.



Activity 1.4: Set Up the Development Environment

The development environment was configured by installing Python and the required libraries.

The project structure was created, and all dependencies were installed before model development.

Steps

- Install Python.
- Install required libraries.
- Create project folders.
- Verify Flask application setup.

```
pip install flask
pip install pandas
pip install numpy
pip install scikit-learn
pip install matplotlib
pip install seaborn
pip install joblib
```

```
✓ HDI-PREDICTOR
  > dataset
  > model
  > screenshots
  > static
  > templates
  > visualizations
  > gitignore
  📄 app.py
  ⓘ README.md
  ≡ requirements.txt
  📄 train_model.py
  📄 visualize.py
```



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

The HDI dataset was cleaned and prepared for Machine Learning. Missing values were handled, required features were selected, and the dataset was divided into training and testing sets. A Linear Regression model was then trained to predict the Human Development Index.

Steps

- Load the HDI dataset.
- Remove missing values.
- Select input features and target variable.
- Split the dataset into training and testing sets.
- Train the Linear Regression model.
- Save the trained model using Joblib.

```
PS C:\Users\dimpl\Downloads\HDI-Predictor\HDI-Predictor> python train_model.py
Life Expected Mean GNI HDI
186 61.7 10.7 3.1 732.0 0.426
187 53.9 8.0 4.3 966.0 0.404
190 55.0 5.5 5.7 768.0 0.385

Rows after cleaning: 3
C:\Users\dimpl\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
warnings.warn(msg, UndefinedMetricWarning)

R2 Score: nan

Model Saved Successfully!
PS C:\Users\dimpl\Downloads\HDI-Predictor\HDI-Predictor>
```

Activity 2.2: Implement the Flask Backend

The Flask backend was developed to load the trained model, accept user inputs, process prediction requests, and display the predicted HDI score through a web interface.

Steps

- Create Flask routes.
- Load the trained model using Joblib.
- Accept user input from the HTML form.
- Predict the HDI score.
- Display the prediction result on the webpage.

```
PS C:\Users\dimpl\Downloads\HDI-Predictor\HDI-Predictor> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 492-382-035
```


Milestone 3: App.py Development

Milestone 3 focuses on developing the main application logic in app.py, which acts as the backbone of the Human Development Index (HDI) Predictor. In this stage, the Flask application is configured to load the trained Machine Learning model, receive user inputs, process prediction requests, and display the predicted HDI score through the web interface.

Activity 3.1: Writing the Main Application Logic in app.py

The app.py file is responsible for connecting the user interface with the trained Machine Learning model. It loads the serialized model using Joblib, accepts input values from the HTML form, performs prediction, and returns the predicted HDI score to the user.

Steps

- Create the Flask application.
- Load the trained model (hdi_model.pkl) using Joblib.
- Define the home route (/) to display the prediction page.
- Accept Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and GNI Per Capita from the user.

- Predict the HDI score using the Linear Regression model.
- Display the prediction result on the webpage.

from flask import Flask, render_template, request

import joblib

app = Flask(__name__)

model = joblib.load("model/hdi_model.pkl")

@app.route("/", methods=["GET", "POST"])

def home():

prediction = None

if request.method == "POST":

values = [[

float(request.form["life"]),

float(request.form["expected"]),

float(request.form["mean"]),

float(request.form["gni"])

]]

prediction = round(model.predict(values)[0], 3)

return render_template("index.html", prediction=prediction)

if __name__ == "__main__":

app.run(debug=True)

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing the user interface of the Human Development Index (HDI) Predictor. The frontend provides a simple and responsive interface that allows users to enter development indicators and view the predicted HDI score in real time.

Activity 4.1: Designing and Developing the User Interface

The frontend was developed using HTML and CSS to provide an easy-to-use interface for entering input values and displaying prediction results.

Steps

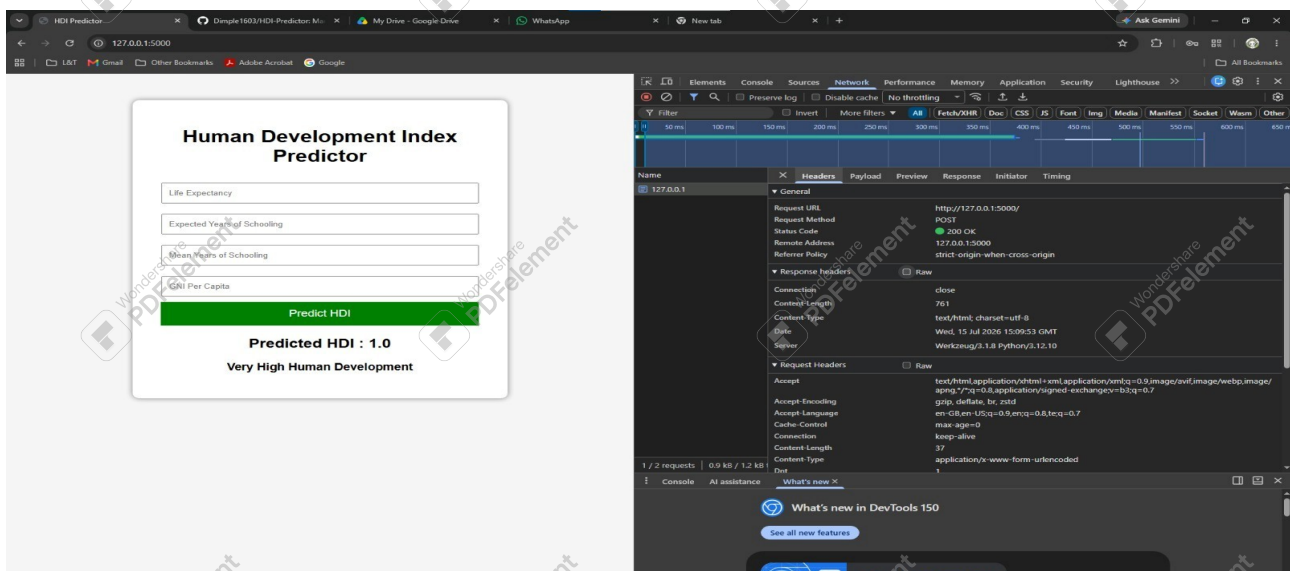
- Create the index.html page.
- Design the input form for HDI indicators.
- Add a Predict button.
- Display the prediction result.
- Apply CSS styling for a clean layout.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

The HTML form was connected to the Flask backend so that user inputs are sent to the prediction model and the predicted HDI score is displayed on the same page.

Steps

- Connect HTML form with Flask routes.
- Submit input values using the POST method.
- Receive prediction from the backend.
- Display the predicted HDI score dynamically.
- Verify the complete prediction workflow.



Milestone 5: Deployment

Milestone 5 focuses on preparing the Human Development Index (HDI) Predictor for execution and verifying that the application functions correctly. The project is deployed locally using the Flask development server and tested to ensure accurate predictions.

Steps

- Open the project folder in Visual Studio Code.
- Install the required dependencies.
- Verify the project structure.
- Ensure the trained model (hdi_model.pkl) and dataset are available.

Activity 5.2: Local Testing and Verification

After configuring the environment, the Flask application was executed locally to verify its functionality and prediction accuracy.

Steps

- Run the Flask application.
- `python app.py`
- Open the browser.
- `http://127.0.0.1:5000`
- Enter sample values.
- Click Predict.
- Verify that the predicted HDI score is displayed correctly.

```
PS C:\Users\dimpl\Downloads\HDI-Predictor\HDI-Predictor> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 492-382-035
```

Exploring the Website's Web Pages

Home / Prediction Page

The Human Development Index (HDI) Predictor provides a simple and user-friendly interface where users can predict the HDI score of a country by entering the required socio-economic indicators. The page contains input fields for Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) Per Capita. After clicking the Predict button, the trained Machine Learning model processes the input and displays the predicted HDI score instantly.

Features

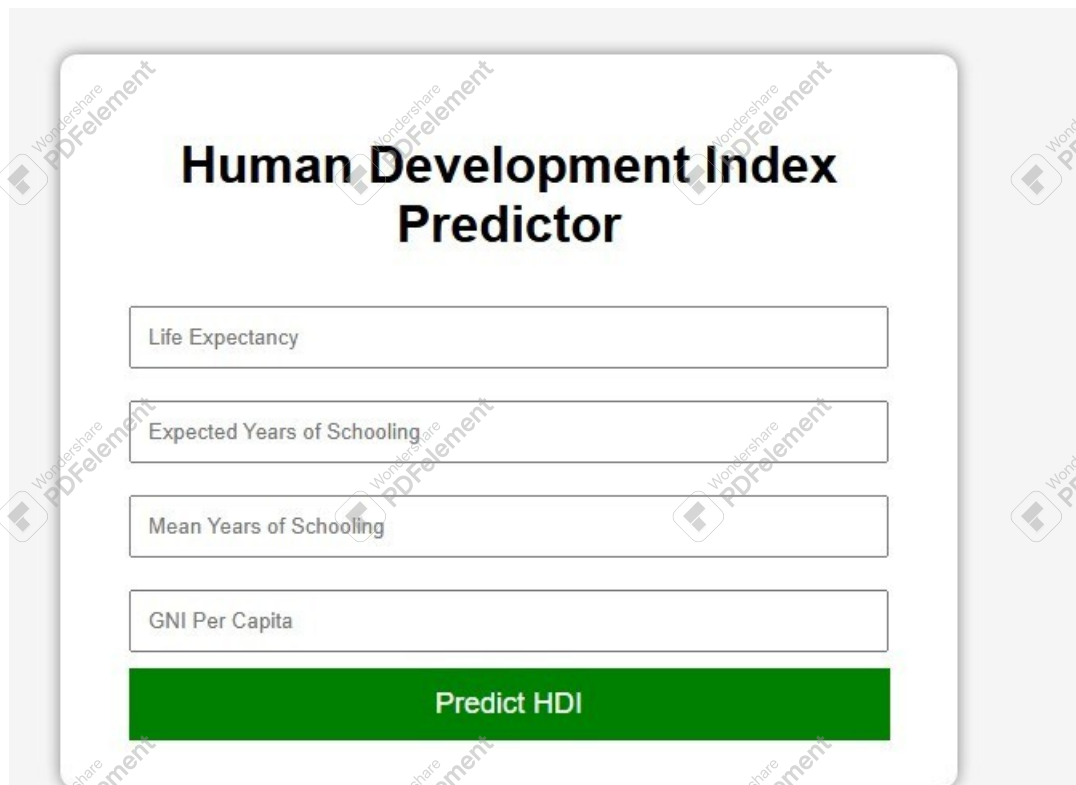
Simple and responsive user interface.

Easy input of HDI indicators.

Real-time prediction.

Displays predicted HDI score.

Flask-powered backend processing.



The screenshot shows a web page titled "Human Development Index Predictor". It features four input fields stacked vertically: "Life Expectancy", "Expected Years of Schooling", "Mean Years of Schooling", and "GNI Per Capita". Below these fields is a prominent green button labeled "Predict HDI". The entire form is centered on a light gray background.

Human Development Index Predictor

Life Expectancy

Expected Years of Schooling

Mean Years of Schooling

GNI Per Capita

Predict HDI

Predicted HDI : 1.0

Very High Human Development

Conclusion

The Human Development Index (HDI) Predictor is a Machine Learning web application developed using Python, Flask, Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn. The project demonstrates the complete Machine Learning lifecycle, including data collection, preprocessing, visualization, model training, evaluation, and deployment through a Flask application.

Using a Linear Regression model, the application predicts the Human Development Index based on important socio-economic indicators such as Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) Per Capita. The system provides quick and accurate predictions through a simple web interface.

Future improvements may include adding multiple Machine Learning models for comparison, improving prediction accuracy, integrating cloud deployment, and expanding the application with interactive dashboards and country-wise analytics.







